

INFORME 3 CATCH & GO

Informe 2 + Testing EP3

Taller Aplicado de Programación – TPY1101
Evaluación Parcial N° 3

Campo	Detalle
Proyecto	Catch & Go – Plataforma de intermediación laboral temporal en Chile
Entregable	Informe consolidado final: análisis, diseño, construcción, plan de pruebas, ejecución QA, evidencias y mejoras
Asignatura	Taller Aplicado de Programación – TPY1101
Equipo	Grupo N° 1
Integrantes	Nicolás Iturrieta Villalobos, Marco Tassara Carrasco y Miguel Arredondo Vargas
Fecha	18 de junio de 2026
Versión	3.0 – Informe completo de presentación EP3

Hoja de control del documento

Ítem	Descripción
Nombre del documento	Informe 3 Catch & Go
Base utilizada	Informe 2 Catch & Go + Informe Testing EP3 Catch & Go
Objetivo	Unificar antecedentes, diseño, desarrollo, testing, mejoras y evidencias en un solo documento formal
Estado	Documento completo para presentación académica
Responsable de consolidación	Equipo Catch & Go

Registro de cambios

Versión	Cambio realizado	Responsable	Fecha
1.0	Informe 2 con problemática, objetivos, alcance, arquitectura y planificación.	Equipo Catch & Go	19/05/2026
2.0	Informe de testing EP3 con plan de pruebas, casos, evidencias y mejoras.	Equipo Catch & Go	18/06/2026
3.0	Unificación y cierre del Informe 3 completo para presentación final.	Equipo Catch & Go	18/06/2026

Índice referencial

- 1. Resumen ejecutivo
- 2. Introducción y contexto del proyecto
- 3. Problemática, oportunidad y justificación
- 4. Equipo de trabajo y metodología
- 5. Objetivos, alcance y entregables del MVP
- 6. Requerimientos funcionales y no funcionales
- 7. Diseño de solución y arquitectura
- 8. Stack tecnológico y modelo de datos
- 9. Planificación, carta Gantt y valorización
- 10. Modelo de negocio y evaluación de factibilidad
- 11. Plan de pruebas de software
- 12. Casos de prueba, trazabilidad y ejecución
- 13. Evidencias de testing
- 14. Mejoras realizadas
- 15. Cumplimiento de calidad, ética y seguridad
- 16. Conclusiones y lecciones aprendidas
- 17. Anexos

1. Resumen ejecutivo

Catch & Go corresponde a una plataforma digital de intermediación laboral temporal orientada al mercado chileno. Su propósito es conectar empresas que necesitan cubrir trabajos rápidos o de corta duración con trabajadores disponibles, considerando aspectos como cercanía geográfica, perfil, habilidades, postulación, reputación y trazabilidad del proceso.

El presente Informe 3 consolida el trabajo desarrollado en las etapas previas del proyecto y agrega el ciclo de validación solicitado para la Evaluación Parcial N° 3. Por lo tanto, no se presenta solo el diseño del producto, sino también el plan de pruebas, la ejecución de casos, las evidencias obtenidas, las mejoras aplicadas y el cumplimiento de criterios de calidad, ética y seguridad.

Durante la fase de construcción se definieron módulos principales como autenticación, perfiles, ofertas laborales, postulaciones, matching, pagos, notificaciones, frontend web y aplicación Android. Para su validación se ejecutó una estrategia de pruebas funcionales, unitarias, de integración, seguridad, usabilidad, persistencia y regresión, cubriendo 72 casos de prueba documentados, de los cuales 71 quedaron aprobados y 1 quedó pendiente o sin marca, según el resumen QA del informe de testing.

El resultado final demuestra que el MVP presenta una base funcional coherente con la problemática planteada: reducir el reclutamiento manual, entregar trazabilidad a las postulaciones y mejorar la confianza entre empresas y trabajadores mediante perfiles, evaluaciones y procesos controlados.

Matriz de cumplimiento de la rúbrica EP3

Criterio de evaluación	Cómo se cumple en este informe	Sección asociada
Plan de pruebas alineado a la problemática	Se presenta una estrategia de pruebas con funcionalidades, acciones, responsables, resultados esperados y criterios de entrada/salida.	Secciones 11 y 12
Aplicación de pruebas de validación	Se documentan pruebas por componente, ejecución de casos, pruebas estándar QA, resultados y evidencias visuales.	Secciones 12 y 13
Mejoras según resultados de pruebas	Se identifican hallazgos y se describen mejoras aplicadas para cumplir estándares de calidad, ética y seguridad.	Sección 14
Informe con evidencias relevantes	Se integran antecedentes del proyecto, tablas, evidencias, capturas y conclusión final.	Todo el documento
Dominio disciplinar	Se fundamenta con criterios de QA, trazabilidad, seguridad, usabilidad, mantenibilidad, datos de prueba y validación por módulos.	Secciones 11 a 15

2. Introducción y contexto del proyecto

El mercado laboral temporal requiere rapidez, confianza y coordinación. En muchos casos, las empresas gestionan reemplazos, turnos o trabajos breves por medios informales, lo que genera pérdida de información, duplicidad de contactos, baja trazabilidad y dificultad para medir desempeño. Catch & Go nace como respuesta a esa necesidad, proponiendo una solución digital que permita publicar ofertas, registrar postulaciones, visualizar perfiles y priorizar candidatos mediante criterios de cercanía y afinidad.

El proyecto se trabajó como un MVP académico, con foco en demostrar los flujos principales antes de escalar a funcionalidades avanzadas. La construcción consideró backend de microservicios, frontend web, aplicación Android, persistencia de datos y mecanismos de autenticación para proteger el acceso.

3. Problemática, oportunidad y justificación

La problemática central identificada es la falta de una herramienta ordenada para administrar trabajos rápidos de 1 a 2 días, especialmente cuando la selección se realiza por canales manuales como WhatsApp, llamadas o redes sociales. Esto impacta tanto a empresas como a trabajadores, ya que se pierde tiempo, no existe historial confiable y la confianza depende de referencias informales.

Tabla 1. Relación entre problemática, efecto y respuesta propuesta

Causa detectada	Efecto en el negocio	Respuesta propuesta por Catch & Go
Reclutamiento manual por WhatsApp, llamadas o redes sociales	Demoras, desorden y baja trazabilidad	Publicación digital de ofertas y postulaciones registradas.
Falta de reputación verificable	Desconfianza entre empresa y trabajador	Perfil, evaluación post-servicio y ranking bidireccional.
Ofertas sin proximidad geográfica	Mayor ausentismo y baja respuesta	Registro de coordenadas y filtros por ubicación.
Procesos tradicionales lentos	Turnos sin cubrir y pérdida operacional	Matching automatizado y dashboard por rol.

A partir de esta necesidad, Catch & Go se justifica como una solución que ordena la oferta y demanda de trabajos temporales, permitiendo registrar cada interacción y generar una base para futuras mejoras, como reputación, ranking, validación documental y recomendaciones inteligentes.

4. Equipo de trabajo y metodología

Tabla 2. Equipo de trabajo y responsabilidades

Integrante	Rol principal	Responsabilidades reales en el proyecto
Nicolás Iturrieta Villalobos	Arquitecto / Product Owner / Backend	Coordinación del alcance, arquitectura técnica, backend, integración, matching inicial y validación de entregables.
Marco Tassara Carrasco	Scrum Master / QA / Android app	Gestión de sprints, pruebas, documentación QA, desarrollo Android, capa de red, login y apoyo Spring Boot.
Miguel Arredondo Vargas	Fullstack / Base de Datos / Frontend	Diseño de base de datos, frontend web, apoyo backend, persistencia, MER y documentación técnica.

La metodología utilizada corresponde a un enfoque ágil basado en Scrum, adaptado al contexto académico y a la disponibilidad parcial del equipo. Se trabajó con backlog de producto, planificación por sprints, revisión de avances, retrospectivas y registro de evidencias para la presentación.

Tabla 3. Artefactos y ritos Scrum aplicados

Artefacto / rito	Aplicación en Catch & Go	Frecuencia
Product Backlog	Historias priorizadas: auth, perfiles, ofertas, matching, web, Android y QA.	Continuo
Sprint Backlog	Tareas semanales por integrante con evidencia esperada.	Semanal
Daily / seguimiento	Qué hice, qué haré, bloqueos; puede ser por WhatsApp/GitHub.	3 veces por semana o diario según carga
Sprint Review	Demostración al docente: software, evidencias, Postman y capturas.	Fin de sprint
Retrospective	Ajuste de alcance, deuda técnica, riesgos y mejoras.	Fin de sprint

5. Objetivos, alcance y entregables del MVP

5.1 Objetivo general

Desarrollar y validar un MVP de plataforma digital para conectar empresas y trabajadores temporales, incorporando registro, autenticación, perfiles, ofertas laborales, postulación, matching inicial y evidencias de calidad mediante un plan de pruebas funcionales.

5.2 Objetivos específicos

- Levantar y documentar la problemática, usuarios objetivo y alcance funcional del producto.
- Diseñar una arquitectura modular que permita separar frontend, backend, datos y componentes móviles.
- Construir los flujos mínimos de autenticación, perfil, ofertas y postulación.
- Definir y ejecutar un plan de pruebas alineado a la problemática del proyecto.
- Aplicar mejoras en función de los hallazgos de QA, asegurando calidad, ética y seguridad.
- Consolidar evidencias en un informe final completo para presentación académica.

5.3 Alcance funcional del MVP

Tabla 4. Módulos principales del MVP

Módulo	Descripción	Estado esperado al 19 de mayo	Prioridad
Autenticación	Registro, login, JWT y control de sesión	Demo funcional con datos de prueba	Alta
Perfiles	Trabajador/empresa, datos base, habilidades, ubicación y rating inicial	CRUD básico y visualización	Alta
Ofertas laborales	Crear, listar, editar y consultar ofertas temporales georreferenciadas	Flujo demostrable web/backend	Alta
Postulaciones	Trabajador postula y empresa revisa candidatos	Tracking base	Alta
Matching automatizado	Scoring por habilidades, distancia, disponibilidad y reputación	Algoritmo inicial documentado	Alta
App web	Dashboards trabajador/empresa, ofertas, perfil, postulaciones	MVP navegable	Alta
App Android	Login, home, design system, red Retrofit y sesión local	MVP demostrable	Media-Alta
Cloud/demo	Frontend, backend y BD desplegados o ambiente cloud de pruebas	Smoke test y evidencias	Alta

6. Requerimientos funcionales y no funcionales

Los requerimientos definidos permiten orientar la construcción y, posteriormente, la validación del producto. En esta etapa, los requisitos se transforman en casos de prueba para asegurar trazabilidad entre lo solicitado, lo desarrollado y lo comprobado.

Tabla 5. Requerimientos funcionales

	Requerimiento	Módulo	Prioridad
RF-01	Registrar trabajadores y empresas con datos mínimos y rol definido.	auth-service / frontend / Android	Alta
RF-02	Iniciar sesión con token JWT y persistir sesión en web/Android.	auth-service / common-jwt / DataStore	Alta
RF-03	Crear y mantener perfil de trabajador/empresa, habilidades, comuna, ubicación y rating.	profile-service	Alta
RF-04	Publicar ofertas temporales con rubro, horario, tarifa, ubicación y descripción.	jobs-service	Alta

RF-05	Postular a una oferta y permitir seguimiento del estado.	jobs-service / frontend	Alta
RF-06	Generar sugerencias de matching por scoring automatizado.	matching-service	Alta
RF-07	Mostrar dashboard diferenciado para trabajador y empresa.	frontend web	Alta
RF-08	Gestionar notificaciones y estados operativos básicos.	Redis / frontend	Media

Tabla 6. Requerimientos no funcionales

Atributo	Indicador	Mecanismo propuesto
Seguridad	Accesos protegidos y datos sensibles controlados	JWT, HTTPS en nube, variables de entorno, CORS y checklist OWASP básico.
Mantenibilidad	Código separado por capas	Controller, Service, Repository, DTO, Mapper, Entity y librerías common.
Portabilidad	Ambiente reproducible	Docker Compose con PostgreSQL 15 y Redis 7.
Escalabilidad	Servicios independientes	Microservicios desacoplados y API Gateway.
Calidad	Pruebas por capa y evidencia	JUnit, Postman, pruebas manuales web, Espresso básico y matriz QA.
Usabilidad	Flujo simple para dos perfiles	Dashboards por rol y onboarding guiado.

7. Diseño de solución y arquitectura

El diseño de solución se plantea mediante una arquitectura modular. Esta decisión facilita el mantenimiento, la prueba independiente de componentes y la evolución futura del sistema. Los módulos principales se comunican por API y se apoyan en persistencia relacional para mantener datos de usuarios, perfiles, ofertas y postulaciones.

Tabla 7. Diagramas recomendados para la defensa

Diagrama	Contenido mínimo	Uso en defensa
Casos de uso	Actor Trabajador, Actor Empresa, Actor Admin; login, crear perfil, publicar oferta, postular, revisar candidatos, evaluar.	Explicar alcance funcional del MVP.
Clases	UserAccount, Profile, JobOffer, JobApplication, MatchSuggestion, Rating, Subscription.	Mostrar modelo lógico de dominio.
Secuencia	Login JWT; publicación de oferta; postulación; cálculo de match.	Demostrar interacción frontend/API/backend/BD.
Componentes	Web App, Android App, API Gateway, microservicios, PostgreSQL, Redis, Supabase/Vercel/Railway.	Defender arquitectura técnica.
Actividad/BPMN	Flujo As-Is manual y flujo To-Be digital con plataforma.	Demostrar mejora del proceso de negocio.

Tabla 8. Wireframes y pantallas mínimas por perfil

Perfil	Pantallas mínimas	Resultado esperado
Público	Landing, quienes somos, contacto, login, registro y selección de rol	Entender propuesta y entrar al sistema.
Trabajador	Dashboard, ofertas, detalle oferta, postular, postulaciones, perfil, ajustes	Encontrar y postular a trabajos cercanos.
Empresa	Dashboard, crear oferta, lista ofertas, candidatos, perfil empresa, suscripción	Publicar necesidades y seleccionar postulantes.
Android	Splash, login, home, ofertas, perfil, estado de postulación	Uso móvil para trabajador.

7.1 Modelo de datos resumido

Tabla 9. Entidades principales del modelo de datos

Entidad	Campos clave	Relaciones
user_accounts	id, name, email, password_hash, role, phone, created_at	1:1 con profile; emite tokens JWT.
profiles	id, user_id, type, rut, skills, commune, latitude, longitude, rating, plan	Pertenece a user_account; se vincula con ofertas/postulaciones.
job_offers	id, company_profile_id, title, description, sector, price, schedule, latitude, longitude, status	Empresa publica múltiples ofertas.

job_applications	id, job_offer_id, worker_profile_id, status, applied_at	Trabajador postula a múltiples ofertas.
match_suggestions	id, job_offer_id, worker_profile_id, score, reason	Resultado del motor de scoring.
ratings	id, from_profile_id, to_profile_id, score, comment	Construye reputación bidireccional.
transactions/subscriptions	id, profile_id, plan, amount, status	Complementario para planes de empresa/trabajador.

8. Stack tecnológico

Tabla 10. Stack tecnológico y uso en el proyecto

Capa	Tecnología real	Evidencia / uso
Backend	Java 21, Spring Boot 3, Spring Cloud Gateway, Maven multi-módulo	api-gateway, auth-service, profile-service, jobs-service, matching-service.
Seguridad	Spring Security + JWT	common-security y common-jwt.
Persistencia	PostgreSQL 15, Spring Data JPA, Flyway	Migraciones db/migration/V*.sql por servicio.
Cache/eventos	Redis 7	Sesiones, notificaciones y apoyo a matching.
Frontend web	Next.js 14, React 18, TypeScript, Tailwind CSS	Dashboards, ofertas, candidatos, perfil, login/registro.
BaaS/ORM	Supabase + Prisma	Autenticación/BD complementaria y schema.prisma.
Android	Kotlin, Jetpack Compose, Hilt, Retrofit, OkHttp, DataStore	Design system, login, capa de red y sesión.
Infraestructura	Docker Compose, Railway, Vercel, Supabase	Entorno local y despliegue académico.
Repositorio	GitHub con GitFlow	main, develop, qa, release/v1.0 y feature/*.

La elección del stack responde a la necesidad de construir un producto demostrable, separando responsabilidades entre componentes. Spring Boot permite implementar servicios backend por dominio, PostgreSQL facilita persistencia relacional, Docker ayuda a reproducir ambientes y las aplicaciones web/Android permiten cubrir diferentes puntos de acceso para usuario empresa y trabajador.

9. Planificación, carta Gantt y valorización

La planificación consideró el período académico desde marzo a junio de 2026. Para representar el avance, se definieron tareas desde inicio y levantamiento de alcance hasta construcción, QA, documentación y preparación de defensa.

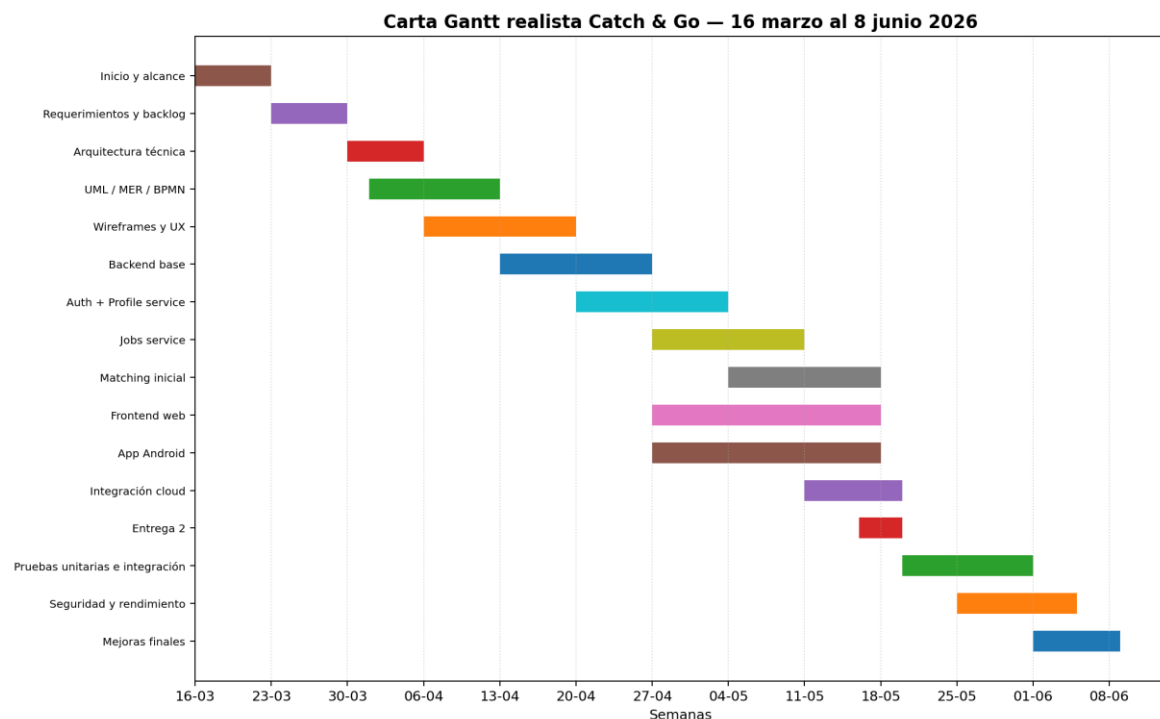


Figura 1. Carta Gantt del proyecto Catch & Go extraída del Informe 2.

Tabla 11. Carta Gantt detallada y valorización por tarea

ID	Tarea	Inicio	Fin	Responsable	Horas	Costo valorizado	Estado
1	Inicio y alcance	16-03-2026	22-03-2026	Todo el equipo	32	\$298.000 CLP	Cerrado
2	Requerimientos y backlog	23-03-2026	29-03-2026	Nicolás	26	\$243.000 CLP	Cerrado
3	Arquitectura técnica	30-03-2026	05-04-2026	Nicolás	26	\$249.000 CLP	Cerrado
4	UML / MER / BPMN	01-04-2026	12-04-2026	Miguel + Marco	24	\$222.000 CLP	Cerrado
5	Wireframes y UX	06-04-2026	19-04-2026	Marco + Miguel	28	\$256.000 CLP	Cerrado
6	Backend base	13-04-2026	26-04-2026	Nicolás	34	\$331.000 CLP	Cerrado
7	Auth + Profile service	20-04-2026	03-05-2026	Nicolás + Miguel	34	\$328.000 CLP	Cerrado
8	Jobs service	27-04-2026	10-05-2026	Miguel + Nicolás	34	\$325.000 CLP	Cerrado
9	Matching inicial	04-05-2026	17-05-2026	Nicolás	34	\$319.000 CLP	Cerrado
10	Frontend web	27-04-2026	17-05-2026	Miguel + Nicolás	48	\$454.000 CLP	En desarrollo
11	App Android	27-04-2026	17-05-2026	Marco	40	\$350.000 CLP	En desarrollo
12	Integración cloud	11-05-2026	19-05-2026	Todo el equipo	30	\$280.000 CLP	Planificado
13	Entrega 2	16-05-2026	19-05-2026	Todo el equipo	24	\$224.000 CLP	Planificado
14	Pruebas unitarias e integración	20-05-2026	31-05-2026	Marco + Miguel	36	\$327.000 CLP	Planificado
15	Seguridad y rendimiento	25-05-2026	04-06-2026	Marco + Nicolás	28	\$257.000 CLP	Planificado
16	Mejoras finales	01-06-2026	08-06-2026	Todo el equipo	30	\$280.000 CLP	Planificado

Tabla 12. Supuestos de costos

Concepto	Criterio usado	Valor estimado
Horas del equipo	508 horas acumuladas entre tres integrantes, con tarifas académicas por rol entre \$8.500 y \$10.000 CLP/hora.	\$4.743.000 CLP
Cloud académico	Uso de Vercel Hobby, Supabase/Railway free trial o planes bajos; bajo tráfico de demostración.	\$0 a \$60.000 CLP/mes aprox.
Herramientas	GitHub, Figma/Draw.io/Lucid free o académico, Android Studio, VS Code, Docker.	\$0 a \$30.000 CLP/mes aprox.
Valor de mercado tercerizado	Equipo externo desarrollando web + backend + Android + BD + QA + despliegue.	\$18.000.000 a \$35.000.000 CLP
MVP educacional presentado	Costo real desembolsado bajo, pero con valor técnico documentado.	\$4.743.000 CLP referencial

10. Modelo de negocio y evaluación de factibilidad

El modelo de negocio se proyecta como una plataforma que puede generar ingresos mediante planes para empresas, comisión por trabajos gestionados y servicios adicionales de validación o reputación. Para el contexto académico, se utilizan supuestos conservadores y se prioriza la validación del problema antes de escalar monetización.

Tabla 13. Proyección referencial del modelo de negocio

Año	Empresas activas	ARPA mensual	Jobs/emp/mes	Ticket prom.	Take rate	Ingresos totales	Costos operativos	Flujo neto
1	25	\$35.000 CLP	4	\$25.000 CLP	8%	\$12.900.000 CLP	\$3.900.000 CLP	\$6.570.000 CLP
2	60	\$40.000 CLP	6	\$27.000 CLP	8%	\$38.131.200 CLP	\$10.900.000 CLP	\$19.878.776 CLP
3	120	\$50.000 CLP	8	\$30.000 CLP	9%	\$103.104.000 CLP	\$23.700.000 CLP	\$57.964.920 CLP
4	200	\$60.000 CLP	10	\$32.000 CLP	9%	\$213.120.000 CLP	\$44.500.000 CLP	\$123.092.600 CLP
5	320	\$70.000 CLP	12	\$35.000 CLP	10%	\$430.080.000 CLP	\$76.500.000 CLP	\$258.113.400 CLP

Tabla 14. Factores de evaluación del proyecto

Factor	Peso	Puntaje	Ponderado	Riesgo	Acción de mejora
Factibilidad técnica	18%	4.2	0.756	Integración incompleta	Reducir alcance a auth + jobs + matching
Factibilidad operativa	15%	4.0	0.600	Baja adopción por complejidad	Onboarding guiado y pruebas con usuarios
Factibilidad económica	15%	3.8	0.570	Supuestos inflados	Validar precios con 5 empresas reales
Factibilidad legal/normativa	12%	3.2	0.384	Riesgo por relación laboral/contratos	Asesoría docente/legal y enfoque marketplace
Seguridad	12%	3.6	0.432	Exposición de credenciales/datos	Variables de entorno y no subir secretos
Calidad y pruebas	12%	3.7	0.444	Errores en defensa	Congelar alcance el 19 de mayo y testear
Escalabilidad	8%	4.0	0.320	Costos cloud suben	Monitoreo y límites de uso
Diferenciación/mercado	8%	4.1	0.328	No lograr masa crítica	Piloto acotado por comuna/rubro

11. Plan de pruebas de software

El plan de pruebas se diseñó para comprobar que el producto construido responde a la problemática planteada y cumple con criterios mínimos de funcionalidad, buenas prácticas, calidad y seguridad. La validación no se limita a revisar pantallas; también considera integración entre componentes, persistencia de datos, manejo de errores y consistencia de los flujos principales.

11.1 Alcance de testing

Tabla 15. Alcance y componentes evaluados

Cliente / usuario objetivo	Público general, empresas y trabajadores temporales de Chile
Producto evaluado	MVP Catch & Go: frontend web, aplicación Android y backend de microservicios
Módulos evaluados	api-gateway, auth-service, profile-service, jobs-service, matching-service, frontend y android-app
Ambiente de pruebas	Ambiente local de desarrollo y QA con ejecución de pruebas unitarias y funcionales
Herramientas utilizadas	JUnit/Gradle, pruebas manuales funcionales, validación visual, planilla de casos de prueba y evidencias en capturas
Criterio de aprobación	Caso aprobado cuando el resultado obtenido coincide con el resultado esperado y no genera errores críticos de seguridad, datos o usabilidad

11.2 Criterios de entrada y salida

Tabla 16. Criterios de entrada y salida del ciclo de pruebas

Tipo	Criterio	Estado
Entrada	Código base disponible en ambiente de desarrollo/QA y módulos principales ejecutables.	Cumplido
Entrada	Planilla de casos de prueba definida y asociada a módulos del sistema.	Cumplido
Entrada	Datos ficticios de prueba disponibles para usuario, empresa, oferta, postulación y pago.	Cumplido
Salida	Ejecución de casos críticos sin errores bloqueantes.	Cumplido
Salida	Evidencias capturadas y asociadas a resultados obtenidos.	Cumplido
Salida	Mejoras aplicadas y registradas en tabla de control.	Cumplido

11.3 Estrategia general de pruebas

Tabla 17. Niveles de prueba aplicados

Nivel de prueba	Componentes	Objetivo	Técnica aplicada	Resultado esperado
Funcional	Web, Android, microservicios	Validar flujos principales del usuario trabajador y empresa.	Ejecución manual guiada por casos CP.	El flujo cumple con el comportamiento definido y guarda datos correctamente.
Unitaria	Frontend, backend, Android	Aislar métodos, componentes y controladores críticos.	JUnit/Gradle y pruebas de componentes.	Cada unidad retorna el resultado esperado sin depender de otros módulos.
Integración	Gateway, servicios y BD	Comprobar comunicación entre módulos y persistencia.	Flujos con datos de prueba y observación de respuestas.	Los servicios responden y se actualiza la información en la base de datos.
Seguridad	auth-service/JWT	Validar credenciales, expiración y manipulación de token.	Pruebas de token válido, alterado y expirado.	El sistema acepta solo tokens válidos y rechaza accesos indebidos.
Usabilidad	Frontend y Android	Validar mensajes, navegación, botones y retorno.	Checklist de pruebas estándar de QA.	La interfaz comunica correctamente los estados y evita confusión.

Calidad operativa	Notificaciones, pagos, errores	Validar manejo de errores y estados de negocio.	Casos negativos, validaciones y simulación de fallas.	El usuario recibe mensajes claros y no se corrompen datos.
-------------------	--------------------------------	---	---	--

11.4 Funcionalidades a comprobar y resultado esperado

Tabla 18. Funcionalidades, acciones y resultados esperados

Funcionalidad	Acción de prueba	Módulo asociado	Resultado esperado	Prioridad
Registro de usuarios	Completar formulario con datos válidos e inválidos.	auth-service / frontend / android-app	Se crean usuarios válidos y se bloquean registros incompletos o con formato incorrecto.	Alta
Inicio de sesión	Ingresar credenciales válidas, inválidas y mantener sesión.	auth-service	Se genera token JWT, se ingresa al dashboard correcto y se rechazan credenciales inválidas.	Alta
Perfiles	Actualizar datos de trabajador y empresa.	profile-service	Los cambios se guardan y se muestran correctamente.	Media
Ofertas laborales	Publicar oferta y validar datos de tarifa/ubicación.	jobs-service	La oferta queda activa y los campos inválidos son bloqueados.	Alta
Postulaciones	Postular, listar y gestionar postulantes.	jobs-service	La postulación cambia de estado de forma correcta.	Alta
Matching	Calcular score y ordenar candidatos.	matching-service	El ranking se genera según distancia y habilidades.	Media
Dashboard y navegación	Validar vistas diferenciadas por rol.	frontend web / android-app	Cada rol ve solo las opciones que le corresponden.	Alta
Notificaciones	Validar alertas de cambio de estado y fallas de servicio.	Redis / frontend	Se despliegan mensajes claros en tiempo real o de error amigable.	Media
Pagos	Iniciar y confirmar transacciones, validar campos.	backend / frontend	Se aceptan datos válidos y se rechazan solicitudes incompletas.	Alta
Seguridad JWT	Generar, leer, alterar y expirar token.	backend	El sistema rechaza tokens manipulados o vencidos.	Alta

12. Base de datos, trazabilidad y ejecución de casos

Para resguardar la ética y la seguridad del proceso de pruebas, los datos utilizados son ficticios. Esto evita exponer información personal real de trabajadores o empresas y permite repetir las pruebas sin afectar usuarios reales.

Tabla 19. Dataset de prueba utilizado

Entidad	Datos de prueba	Uso en pruebas	Control de seguridad/ética
Trabajador	Juan Pérez, juan.perez@email.com, +56912345678, Password123!	Registro, login, perfil, postulación y dashboard trabajador.	Usuario ficticio, sin datos personales reales.
Empresa	Empresa Corp, contacto@empresa.com, +56987654321, CompanyPass!	Registro empresa, publicación de oferta, gestión de candidatos.	Empresa ficticia para evitar uso de datos comerciales reales.
Oferta	Auxiliar de Bodega, Logística, Maipú, \$4.000/hr, ubicación lat/Ing	Publicación, listado, postulación y matching.	Datos de ubicación referenciales y no sensibles.
Postulación	ID oferta 101, ID trabajador 5, estado Pendiente/Aceptado/Rechazado	Validación de cambios de estado y notificaciones.	Estados controlados en ambiente de prueba.
Pago	user-123, monto 10.000, token real-token-123	Inicio y confirmación de pago en controlador.	Token simulado; no se utiliza información bancaria real.
JWT	Subject user@test.com, expiración 3600 segundos, token alterado y expirado	Generación, claims, expiración y tampering.	No se exponen secretos reales en el informe.

12.1 Matriz de trazabilidad

Tabla 20. Trazabilidad entre requisitos y casos de prueba

Requisito	Descripción	Casos de prueba asociados
RF-01	Registro de usuarios trabajador/empresa con validaciones de campos obligatorios, formato de correo y contraseña.	CP-01, CP-02, CP-03, CP-04

RF-02	Inicio de sesión, generación de token JWT, persistencia y control de credenciales inválidas.	CP-05, CP-06, CP-07, CP-08, CP-65, CP-66, CP-67, CP-68
RF-03	Mantenimiento de perfiles, reputación inicial y datos de trabajador/empresa.	CP-09, CP-10, CP-11
RF-04	Publicación de ofertas laborales, validación de tarifa y disponibilidad para trabajadores.	CP-12, CP-13
RF-05	Postulación, seguimiento y gestión de postulantes por parte de empresa.	CP-14, CP-15, CP-16
RF-06	Motor de matching por habilidades, ubicación y visualización de candidatos recomendados.	CP-17, CP-18
RF-07	Dashboards diferenciados por rol, navegación, notificaciones y frontend web.	CP-19, CP-20, CP-21, CP-22, CP-23, CP-24, CP-55, CP-56, CP-57, CP-58, CP-59, CP-60, CP-61, CP-62, CP-63, CP-64
RF-08	Módulos complementarios: pagos, membresías, documentos, seguridad de JWT y app Android.	CP-25, CP-26, CP-27, CP-28, CP-29, CP-30, CP-31, CP-32, CP-33, CP-34, CP-35, CP-36, CP-37, CP-38, CP-39, CP-40, CP-41, CP-42, CP-43, CP-44, CP-45, CP-46, CP-47, CP-48, CP-49, CP-50, CP-51, CP-52, CP-53, CP-54, CP-69, CP-70, CP-71, CP-72

12.2 Resumen ejecutivo de ejecución

Tabla 21. Resumen de ejecución QA

Total de casos de prueba funcionales/unitarios	72
Casos OK	71
Casos pendientes / sin marca	1
Pruebas estándar QA aplicadas	17
Pruebas estándar OK	14
Pruebas estándar NA	3
Cobertura por componentes	auth-service: 8, profile-service: 3, jobs-service: 5, matching-service: 2, frontend web: 2, Redis / frontend: 2, frontend: 7, android-app: 35, backend: 8

El resultado global muestra una cobertura amplia del producto, con 72 casos de prueba documentados. La mayoría de los casos fue aprobada, lo que permite afirmar que el producto cumple con una base funcional estable para una presentación académica de MVP.

12.3 Tabla completa de casos de prueba

La siguiente tabla consolida los casos de prueba funcionales, unitarios y de integración definidos para validar los componentes del MVP. Se mantiene el formato de caso, módulo, tipo, funcionalidad, acción y resultado esperado/obtenido para facilitar la revisión docente.

Tabla 22. Casos de prueba del proyecto Catch & Go

CP	Módulo	Tipo	Funcionalidad a comprobar	Acción a realizar	Resultado esperado / obtenido
CP-01	auth-service	Funcional	Registro de Trabajador con datos correctos	1. Ingresar al formulario de registro en Web o App Android. 2. Ingresar Nombre, Correo, Teléfono, Contraseña, seleccionar rol 'Trabajador'. 3. Presionar el botón 'Registrar'.	Se muestra la pantalla de registro con todos los campos. Los campos se completan correctamente sin errores visuales. Se registra el usuario en BD, se redirige al Login y se muestra un mensaje de éxito. Resultado: OK
CP-02	auth-service	Funcional	Registro de Empresa con datos correctos	1. Ingresar al formulario de registro en Web. 2. Ingresar Nombre, Correo, Teléfono, Contraseña, seleccionar rol 'Empresa'. 3. Presionar el botón 'Registrar'.	Se muestra la pantalla de registro. Los campos se completan correctamente. Registro exitoso de la cuenta de empresa en BD y redirección al login. Resultado: OK
CP-03	auth-service	Funcional	Registro fallido por campos vacíos	1. Ingresar al formulario de registro. 2. Dejar campos en blanco y presionar 'Registrar'.	Se visualiza el formulario. Se muestran alertas visuales indicando que los campos son obligatorios y el registro se bloquea. Resultado: OK
CP-04	auth-service	Funcional	Registro fallido por formato de correo inválido	1. Ingresar al formulario de registro. 2. Ingresar correo sin formato válido (ej: 'correo.com') y completar los demás datos. 3. Presionar 'Registrar'.	Se abre el formulario. El sistema indica que el formato de correo es incorrecto en tiempo real. No se realiza el registro y se mantiene en la misma pantalla. Resultado: OK
CP-05	auth-service	Funcional	Inicio de sesión de Trabajador exitoso	1. Ingresar al Login en la Web o App Android. 2. Ingresar correo y contraseña válidos de Trabajador. 3. Presionar 'Iniciar Sesión'.	Se despliega pantalla de login. Credenciales escritas correctamente. Inicio de sesión exitoso, se genera y guarda el token JWT, y se ingresa al Dashboard de Trabajador. Resultado: OK
CP-06	auth-service	Funcional	Inicio de sesión de Empresa exitoso	1. Ingresar al Login de la Web. 2. Ingresar correo y contraseña de Empresa. 3. Presionar 'Iniciar Sesión'.	Se muestra el login. Datos completados. Ingreso exitoso al Dashboard de Empresa con token JWT activo en cabeceras. Resultado: OK
CP-07	auth-service	Funcional	Inicio de sesión fallido por credenciales inválidas	1. Ingresar al Login. 2. Ingresar contraseña incorrecta o correo no registrado. 3. Presionar 'Iniciar Sesión'.	Se muestra el login. Se ingresan los datos de prueba. Se muestra el mensaje 'Credenciales incorrectas' y se bloquea el acceso. Resultado: OK
CP-08	auth-service	Funcional	Persistencia de sesión (DataStore / LocalStorage)	1. Iniciar sesión correctamente en la App Android o Web. 2. Cerrar la aplicación o refrescar la página. 3. Volver a abrir la aplicación / Web.	Inicio de sesión exitoso. Aplicación cerrada / pestaña recargada. Se mantiene la sesión abierta y redirige directamente al Dashboard sin pedir login. Resultado: OK
CP-09	profile-service	Funcional	Actualizar perfil de Trabajador	1. Ir a 'Mi Perfil' en la App Android o Web. 2. Editar habilidades, comuna y presionar 'Guardar'.	Se muestra la pantalla del perfil. Se actualizan los datos exitosamente en la base de datos y se muestra mensaje de confirmación. Resultado: OK
CP-10	profile-service	Funcional	Actualizar perfil de Empresa	1. Ir a 'Perfil Empresa' en el Frontend Web. 2. Modificar dirección de la empresa y teléfono de contacto, presionar 'Guardar'.	Se despliega la pantalla de edición. Los cambios se guardan exitosamente y se actualiza la información visualizada en el perfil. Resultado: OK
CP-11	profile-service	Funcional	Asignación de reputación (Rating) inicial	1. Registrar un nuevo usuario (trabajador o empresa). 2. Consultar el perfil recién creado.	Registro de usuario completado. El perfil se inicializa con un rating predeterminado de 5.0. Resultado: OK
CP-12	jobs-service	Funcional	Publicar oferta de trabajo exitosamente	1. Iniciar sesión como Empresa e ir a 'Publicar Oferta'. 2. Ingresar título, rubro, horario, tarifa por hora, ubicación y descripción. 3. Presionar 'Publicar'.	Se muestra el formulario de publicación. Campos ingresados correctamente. La oferta se crea en estado 'Activa' y se hace visible para los trabajadores en su dashboard. Resultado: OK
CP-13	jobs-service	Funcional	Publicar oferta fallida por tarifa inválida	1. Ir al formulario de 'Publicar Oferta'. 2. Completar datos con tarifa negativa o igual a cero. 3. Presionar 'Publicar'.	Se despliega el formulario. Se ingresa el valor negativo. El sistema muestra un mensaje de validación y bloquea el botón de publicar. Resultado: OK

CP-14	jobs-service	Funcional	Postulación exitosa a oferta laboral	1. Iniciar sesión como Trabajador. 2. Buscar una oferta de trabajo activa y presionar 'Postular'.	Acceso al listado de ofertas. La postulación se registra en la base de datos con estado 'Pendiente' y el botón cambia a 'Postulado'. Resultado: OK
CP-15	jobs-service	Funcional	Visualización de postulaciones del Trabajador	1. Iniciar sesión como Trabajador e ir a 'Mis Postulaciones'. 2. Revisar el estado de cada postulación.	Se muestra la pantalla de postulaciones. Se listan las ofertas postuladas con su estado (Pendiente, Aceptado o Rechazado) actualizado. Resultado: OK
CP-16	jobs-service	Funcional	Gestión de postulantes por la Empresa	1. Iniciar sesión como Empresa e ir al detalle de una oferta publicada. 2. Ver lista de candidatos y presionar 'Aceptar' o 'Rechazar' en uno de ellos.	Se visualiza la lista de candidatos con sus datos básicos. El estado del candidato cambia a 'Aceptado' / 'Rechazado' en pantalla y se actualiza en la BD. Resultado: OK
CP-17	matching-service	Funcional	Cálculo de scoring por matching automatizado	1. Crear una oferta con requisitos específicos y ubicación. 2. Postular con un trabajador cuyas habilidades y ubicación coincidan parcialmente. 3. Verificar el valor de score asignado por el sistema en la base de datos.	Oferta y postulación creadas. Postulación registrada. El motor calcula un score de compatibilidad (ej. 85%) basado en distancia y habilidades coincidentes. Resultado: OK
CP-18	matching-service	Funcional	Visualización de sugerencias de trabajadores recomendados	1. Iniciar sesión como Empresa. 2. Ingresar a la sección de 'Candidatos Recomendados' de una oferta.	Carga del panel de candidatos. Los candidatos se muestran ordenados de mayor a menor según su matching score de forma descendente. Resultado: OK
CP-19	frontend web	Funcional	Acceso a Dashboard de Trabajador	1. Iniciar sesión como Trabajador. 2. Verificar las secciones visibles en el dashboard principal.	Inicio de sesión exitoso. Se visualizan ofertas sugeridas, mapa de geolocalización y postulaciones activas. No se muestran opciones de empresa. Resultado: OK
CP-20	frontend web	Funcional	Acceso a Dashboard de Empresa	1. Iniciar sesión como Empresa. 2. Verificar las secciones visibles en el panel.	Inicio de sesión exitoso. Se visualizan opciones de 'Publicar nueva oferta', 'Lista de ofertas publicadas' e informes de postulantes activos. Resultado: OK
CP-21	Redis / frontend	Funcional	Notificación de cambio de estado de postulación	1. Empresa acepta la postulación de un Trabajador. 2. Trabajador se encuentra navegando en la aplicación. 3. Verificar la llegada de la notificación en tiempo real.	La empresa cambia el estado. El trabajador sigue activo en la sesión. Se despliega una alerta visual instantánea en la pantalla del trabajador informando que fue aceptado. Resultado: OK
CP-22	Redis / frontend	Funcional	Alertas de sistema y estados operativos	1. Provocar un error simulado en el Matching Service o Jobs Service. 2. Verificar que se desplieguen alertas de mantenimiento o reintento en el frontend.	Interrupción de servicio. El frontend detecta la falla de red y muestra un mensaje amigable indicando problemas temporales. Resultado: OK
CP-23	frontend	Prueba Unitarias	Prueba unitaria de Barra de Navegación (Navbar)	1. Renderizar Navbar en estado sin sesión. 2. Renderizar Navbar con sesión activa de Trabajador o Empresa. 3. Ejecutar flujo de cierre de sesión.	Se visualizan enlaces de Quiénes Somos, Contacto, Términos y botones de ingreso. Se visualizan opciones de dashboard específicas por rol y campana de notificaciones. Se limpian claves de localStorage y se redirige a /login. Resultado: OK
CP-24	frontend	Prueba Unitarias	Prueba unitaria de Campana de Notificaciones	1. Renderizar campana con unreadCount > 0. 2. Hacer clic en campana y desplegar listado. 3. Ejecutar limpieza o marcado de lectura.	Se dibuja un círculo rojo con el número de notificaciones no leídas. Se abre el panel mostrando las notificaciones y botones de Limpiar/Marcar como leída. Se invocan las funciones del contexto clearAll o markAsRead correspondientes. Resultado: OK
CP-25	frontend	Prueba Unitarias	Prueba unitaria de Modal de Pago (PaymentGatewayModal)	1. Renderizar modal en transferencia manual sin archivo adjunto. 2. Adjuntar comprobante de pago en transferencia manual. 3. Cambiar a pasarela Webpay Plus y presionar pagar.	El botón Confirmar Pago está deshabilitado. El botón Confirmar Pago se habilita y al pulsar llama a onSubmit con los datos. Se invoca la función onPayWithWebpay y el botón muestra estado de carga. Resultado: OK
CP-26	frontend	Prueba Unitarias	Prueba unitaria de Página de Login (page.tsx)	1. Intentar iniciar sesión con campos vacíos. 2. Iniciar sesión exitosamente mediante API. 3. Iniciar sesión con error de credenciales.	Se despliegan mensajes locales indicando que los campos son obligatorios. Guarda auth_token e user_info en localStorage y redirige al perfil correspondiente. Se despliega el mensaje de error retornado por la API de autenticación. Resultado: OK
CP-27	android-app	Prueba Unitarias	Prueba unitaria de inicialización del registro (RegisterViewModel)	1. Inicializar el RegisterViewModel con dependencias mockeadas en JUnit.	El estado inicial del formulario está completamente vacío y el flag canSubmit está en false. Resultado: OK

CP-28	android-app	Prueba Unitarias	Prueba unitaria de validación de campos del formulario de registro	1. Simular ingreso de nombre, correo, rut, teléfono, contraseña de 11 caracteres y selección de rol.	El flag canSubmit cambia a true, permitiendo el envío del formulario. Resultado: OK
CP-29	android-app	Prueba Unitarias	Prueba unitaria de flujo de registro exitoso	1. Cargar campos válidos en el RegisterViewModel. 2. Mockear registro exitoso en AuthRepository. 3. Ejecutar onSubmit().	Se llama a authRepository.register y el estado final isLoading queda en false sin errores. Resultado: OK
CP-30	android-app	Prueba Unitarias	Prueba unitaria de gestión de errores de red en el registro	1. Cargar campos válidos. 2. Mockear excepción IOException de red en AuthRepository.register. 3. Ejecutar onSubmit().	Se llama a authRepository.register, el estado final isLoading queda en false y se asigna el mensaje de error de conexión. Resultado: OK
CP-31	android-app	Prueba Unitarias	Prueba unitaria de cálculo de distancia geográfica (MatchEngine)	1. Invocar MatchEngine.calculateDistance con dos coordenadas conocidas en Santiago.	Retorna la distancia en kilómetros con precisión matemática (~4.6 km). Resultado: OK
CP-32	android-app	Prueba Unitarias	Prueba unitaria de distancia geográfica ante coordenadas nulas	1. Invocar MatchEngine.calculateDistance con valores de coordenadas nulos.	El motor de emparejamiento retorna -1.0 indicando error o datos insuficientes. Resultado: OK
CP-33	android-app	Prueba Unitarias	Prueba unitaria de completitud de perfil al 100% (MatchEngine)	1. Invocar MatchEngine.calculateProfileCompletion con un perfil completamente poblado.	El motor de emparejamiento retorna 100 indicando perfil al máximo. Resultado: OK
CP-34	android-app	Prueba Unitarias	Prueba unitaria de completitud de perfil nulo u vacío	1. Invocar MatchEngine.calculateProfileCompletion con un perfil nulo o vacío.	El motor retorna 0 indicando que el perfil no posee avances de edición. Resultado: OK
CP-35	android-app	Prueba Unitarias	Prueba unitaria de relevancia básica de matching con plan TRIAL	1. Crear perfiles e incentivos que coincidan. 2. Invocar MatchEngine.calculate con plan TRIAL.	El motor de emparejamiento retorna el score de relevancia sin bonificación de plan premium (score >= 80). Resultado: OK
CP-36	android-app	Prueba Unitarias	Prueba unitaria de relevancia premium de matching con plan PREMIUM	1. Invocar MatchEngine.calculate con plan PREMIUM y los mismos datos de CP-35.	El motor aplica la bonificación de relevancia del 20% sobre el puntaje base, retornando el puntaje premium. Resultado: OK
CP-37	android-app	Prueba Unitarias	Prueba unitaria de inicialización de publicación (CrearOfertaViewModel)	1. Inicializar el CrearOfertaViewModel con dependencias mockeadas en JUnit.	El estado inicial del formulario está completamente vacío con valores por defecto. Resultado: OK
CP-38	android-app	Prueba Unitarias	Prueba unitaria de modificación de campos de publicación	1. Simular ingreso de título, descripción, ubicación, categoría, remuneración y fechas.	El estado interno del ViewModel se actualiza correctamente reflejando las entradas del usuario. Resultado: OK
CP-39	android-app	Prueba Unitarias	Prueba unitaria de campos obligatorios faltantes en publicación	1. Rellenar campos omitiendo la fecha de inicio. 2. Ejecutar submit().	Se interrumpe la ejecución de publicación y se asigna el mensaje de error local indicando completar campos obligatorios. Resultado: OK
CP-40	android-app	Prueba Unitarias	Prueba unitaria de publicación exitosa de oferta laboral	1. Cargar campos válidos. 2. Mockear sesión de empresa y respuesta de éxito en JobsApi.create. 3. Ejecutar submit().	Se llama a jobsApi.create y el estado final success queda en true sin errores. Resultado: OK
CP-41	android-app	Prueba Unitarias	Prueba unitaria de fallo en la API al publicar oferta	1. Cargar campos válidos. 2. Mockear una excepción IOException en la llamada a JobsApi.create. 3. Ejecutar submit().	La excepción es capturada, el estado final success queda en false y se despliega el mensaje de error de la API. Resultado: OK
CP-42	android-app	Prueba Unitarias	Prueba unitaria de mapeo de SocketTimeoutException (ErrorMapper)	1. Invocar ErrorMapper.map pasando una instancia de SocketTimeoutException.	El mapeador retorna ApiError.Timeout con el mensaje 'La solicitud demoró demasiado'. Resultado: OK
CP-43	android-app	Prueba Unitarias	Prueba unitaria de mapeo de IOException (ErrorMapper)	1. Invocar ErrorMapper.map pasando una instancia de IOException.	El mapeador retorna ApiError.NoConnection con el mensaje 'Sin conexión a internet'. Resultado: OK
CP-44	android-app	Prueba Unitarias	Prueba unitaria de mapeo de excepciones genéricas (ErrorMapper)	1. Invocar ErrorMapper.map con una RuntimeException genérica.	El mapeador retorna ApiError.Unknown conservando el mensaje original de la excepción. Resultado: OK
CP-45	android-app	Prueba Unitarias	Prueba unitaria de mapeo de error HTTP 401 (ErrorMapper)	1. Mockear HttpException con código 401. 2. Invocar ErrorMapper.map.	El mapeador retorna ApiError.Unauthorized con el mensaje de sesión expirada. Resultado: OK
CP-46	android-app	Prueba Unitarias	Prueba unitaria de mapeo de error HTTP 404 (ErrorMapper)	1. Mockear HttpException con código 404. 2. Invocar ErrorMapper.map.	El mapeador retorna ApiError.NotFound con el mensaje de recurso no encontrado. Resultado: OK
CP-47	android-app	Prueba Unitarias	Prueba unitaria de carga de detalles de oferta (OfferDetailViewModel)	1. Inicializar el OfferDetailViewModel con el ID 101.	El estado del ViewModel almacena los datos de la oferta y isLoading se establece en false. Resultado: OK

				2. Mockear respuesta de éxito en <code>jobsRepository.getOffer(101)</code> .	
CP-48	android-app	Prueba Unitarias	Prueba unitaria de fallo al cargar la oferta	1. Mockear excepción de red en <code>jobsRepository.getOffer(102)</code> . 2. Inicializar el <code>ViewModel</code> con ID 102.	Se cancela la carga, <code>isLoading</code> queda en false y el mensaje de error de red se mapea correctamente en <code>errorMessage</code> . Resultado: OK
CP-49	android-app	Prueba Unitarias	Prueba unitaria de postulación exitosa a oferta	1. Invocar <code>submitApplication</code> con un mensaje de presentación en el <code>ViewModel</code> .	Se llama a <code>applicationsRepository.apply()</code> , <code>isApplying</code> queda en false, y no se presentan errores. Resultado: OK
CP-50	android-app	Prueba Unitarias	Prueba unitaria de postulación fallida por error del servidor	1. Mockear excepción <code>IOException</code> en <code>applicationsRepository.apply()</code> . 2. Invocar <code>submitApplication</code> .	<code>isApplying</code> queda en false y <code>applyError</code> se actualiza con la descripción de error de conexión traducida. Resultado: OK
CP-51	android-app	Prueba Unitarias	Prueba unitaria de cancelación exitosa de postulación	1. Invocar <code>cancelPostulation</code> con el ID de postulación 999.	Se llama a <code>cancelApplication</code> y se actualizan los estados correctamente sin errores. Resultado: OK
CP-52	android-app	Prueba Unitarias	Prueba unitaria de estado inicial por defecto (<code>NotificationsViewModel</code>)	1. Inicializar el <code>NotificationsViewModel</code> . 2. Comprobar que la lista de notificaciones esté vacía, el estado de conexión sea false y el conteo de no leídas sea 0.	<code>notifications</code> es vacío, <code>isConnected</code> es false y <code>unreadCount</code> es 0. Resultado: OK
CP-53	android-app	Prueba Unitarias	Prueba unitaria de cálculo de notificaciones no leídas (<code>NotificationsViewModel</code>)	1. Simular una lista de notificaciones con 2 no leídas y 1 leída. 2. Actualizar el <code>StateFlow</code> del repositorio. 3. Comprobar que <code>unreadCount</code> se actualice a 2.	El valor expuesto por <code>unreadCount</code> es exactamente 2. Resultado: OK
CP-54	android-app	Prueba Unitarias	Prueba unitaria de marcar notificación como leída (<code>NotificationsViewModel</code>)	1. Invocar <code>markAsRead("alerta_99")</code> en el <code>ViewModel</code> . 2. Verificar que se invoque <code>markAsRead("alerta_99")</code> en el repositorio.	Se invoca a <code>notificationRepository.markAsRead("alerta_99")</code> exactamente 1 vez. Resultado: OK
CP-55	android-app	Prueba Unitarias	Prueba unitaria de limpiar notificaciones no leídas (<code>NotificationsViewModel</code>)	1. Cargar lista de notificaciones con pendientes. 2. Invocar <code>clearUnread()</code> 3. Verificar llamadas a <code>markAsRead</code> en el repositorio solo para las no leídas.	Se invoca a <code>markAsRead()</code> en el repositorio solo para los IDs de notificaciones no leídas. Resultado: OK
CP-56	android-app	Prueba Unitarias	Prueba unitaria de limpiar todas las notificaciones (<code>NotificationsViewModel</code>)	1. Invocar <code>clearAll()</code> en el <code>ViewModel</code> . 2. Verificar que se llame a <code>clearAll()</code> en el repositorio.	Se invoca a <code>notificationRepository.clearAll()</code> exactamente 1 vez. Resultado: OK
CP-57	android-app	Prueba Unitarias	Prueba unitaria de carga inicial del feed y completitud de perfil (<code>FeedViewModel</code>)	1. Inicializar el <code>FeedViewModel</code> con sesión activa. 2. Mockear <code>getProfile()</code> con perfil completo. 3. Comprobar que las ofertas se carguen y <code>profileCompletion</code> sea mayor a 0.	<code>isLoading</code> es false, <code>offers</code> contiene los datos iniciales y <code>profileCompletion</code> se calcula correctamente. Resultado: Pendiente de validación
CP-58	android-app	Prueba Unitarias	Prueba unitaria de filtrado dinámico por categoría (<code>FeedViewModel</code>)	1. Invocar <code>onCategorySelect(JobCategory.CONSERJE)</code> en el <code>ViewModel</code> . 2. Verificar que se invoque <code>observeOffers</code> con el filtro de categoría correcto. 1. Cargar múltiples ofertas en el flujo.	<code>observeOffers</code> es llamado con el <code>JobFilter</code> de la categoría y el estado se actualiza. Resultado: OK
CP-59	android-app	Prueba Unitarias	Prueba unitaria de búsqueda textual de ofertas en tiempo real (<code>FeedViewModel</code>)	2. Invocar <code>onSearchChange("Conserje")</code> en el <code>ViewModel</code> . 3. Comprobar que la lista visible de ofertas filtre y muestre solo las coincidentes. 1. Cargar postulaciones con estados mixtos.	La lista filtrada contiene únicamente las ofertas que coinciden con el texto de búsqueda. Resultado: OK
CP-60	android-app	Prueba Unitarias	Prueba unitaria de filtrado por pestañas de postulaciones y conteos (<code>FeedViewModel</code>)	2. Cambiar a pestañas <code>POSTULACIONES</code> y <code>PAGOS</code> . 3. Comprobar el filtro de postulaciones y la coincidencia de los contadores.	El estado actualiza <code>activeTab</code> , filtra correctamente la lista mostrada y calcula los contadores de cada sección. Resultado: OK
CP-61	android-app	Prueba Unitarias	Prueba unitaria de refresco manual del feed (<code>FeedViewModel</code>)	1. Invocar <code>onRefresh()</code> en el <code>ViewModel</code> . 2. Verificar llamadas de refresco en los repositorios de ofertas y postulaciones. 1. Cargar campos válidos en el formulario de registro.	Se llaman a <code>jobsRepository.refresh()</code> y <code>applicationsRepository.refreshFromBackend()</code> para sincronizar con el backend. Resultado: OK
CP-62	frontend	Prueba Unitarias	Prueba de validación de fortaleza de contraseña (<code>RegisterPage</code>)	2. Escribir contraseña débil (sin mayúsculas o números). 3. Hacer click en Crear Cuenta. 1. Cargar campos válidos.	El formulario se bloquea y muestra el mensaje de error: 'Mínimo 8 caracteres, 1 mayúscula y 1 número'. Resultado: OK
CP-63	frontend	Prueba Unitarias	Prueba de coincidencia de contraseñas (<code>RegisterPage</code>)	2. Escribir contraseñas diferentes en los campos Contraseña y Confirmar. 3. Hacer click en Crear Cuenta. 1. Cargar todos los campos válidos de registro.	El formulario se bloquea y muestra el mensaje de error: 'Las contraseñas no coinciden'. Resultado: OK

CP-64	frontend	Prueba Unitarias	Prueba de flujo completo de registro exitoso tras aceptar términos (RegisterPage)	2. Hacer click en Crear Cuenta. 3. En el modal de términos, hacer click en Aceptar y Registrarse. 4. Mockear respuesta exitosa de authApi.register.	Se realiza el registro en la API, se almacena el token JWT y los datos de sesión en localStorage, y se redirige a /trabajador/perfil. Resultado: OK
CP-65	backend	Prueba Unitarias	Prueba de generación exitosa de JWT (JwtService)	1. Invocar generateToken pasando un subject y tiempo de expiración. 2. Comprobar que la respuesta no sea nula y tenga el formato JWT correcto. 1. Generar un token JWT válido con un subject y expiración específicos.	Se genera un string JWT compacto no vacío con tres secciones (header, payload y firma) separadas por puntos. Resultado: OK
CP-66	backend	Prueba Unitarias	Prueba de lectura correcta de Claims (JwtService)	2. Decodificar el token con parseClaims. 3. Comprobar que los claims coincidan. 1. Generar un token JWT válido.	El método parseClaims extrae exitosamente el subject y el tiempo de expiración correcto del token decodificado. Resultado: OK
CP-67	backend	Prueba Unitarias	Prueba de detección de alteración/tampering en token (JwtService)	2. Alterar un caracter del token para invalidar su firma. 3. Invocar parseClaims con el token alterado.	El método parseClaims detecta la alteración de la firma y lanza una excepción de tipo SignatureException. Resultado: OK
CP-68	backend	Prueba Unitarias	Prueba de control de expiración de token (JwtService)	1. Generar un token JWT con tiempo de expiración en el pasado. 2. Invocar parseClaims con el token expirado.	El método parseClaims lanza una excepción de tipo ExpiredJwtException indicando que el token ya no es válido. Resultado: OK
CP-69	backend	Prueba Unitarias	Prueba de validación de parámetros faltantes al iniciar pago (WebpayController)	1. Enviar una petición POST a /profiles/webpay/init omitiendo el parámetro amount. 2. Comprobar la respuesta HTTP. 1. Enviar una petición POST a /profiles/webpay/init con todos los parámetros correctos.	Retorna HTTP 400 Bad Request y un JSON indicando que los campos userId, amount y returnUrl son requeridos. Resultado: OK
CP-70	backend	Prueba Unitarias	Prueba de inicialización exitosa de transacción (WebpayController)	2. Mockear respuesta exitosa del servicio Webpay. 3. Comprobar la respuesta HTTP.	Retorna HTTP 200 OK y una respuesta JSON que contiene el token de transacción y la URL de redirección. Resultado: OK
CP-71	backend	Prueba Unitarias	Prueba de validación de token faltante al confirmar pago (WebpayController)	1. Enviar una petición POST a /profiles/webpay/confirm con payload vacío o sin el token. 2. Comprobar la respuesta HTTP. 1. Enviar una petición POST a /profiles/webpay/confirm con un token válido.	Retorna HTTP 400 Bad Request y un mensaje de error indicando que el token es requerido. Resultado: OK
CP-72	backend	Prueba Unitarias	Prueba de confirmación exitosa de transacción (WebpayController)	2. Mockear respuesta exitosa del servicio confirmando la transacción. 3. Comprobar la respuesta HTTP.	Retorna HTTP 200 OK y una respuesta JSON indicando que el estado de la transacción es AUTHORIZED. Resultado: OK

13. Aplicación de pruebas, resultados y evidencias

13.1 Pruebas estándar de QA

Tabla 23. Validaciones estándar aplicadas

ID	Validación estándar	Estado
E-1.1	Verificar que se encuentre un icono en todos los mensajes desplegados por la aplicación	OK
E-1.2	Verificar que los botones tengan un icono o frase adecuado a su función.	OK
E-1.3	Verificar que las aplicaciones con múltiples pantallas exista la opción “Retornar” a la pantalla anterior.	OK
E-2.1	Verificar correcta lectura de la información ingresada.	OK
E-2.2	Verificar que no se puedan hacer modificaciones en las pantallas de Consultas	OK
E-2.3	Verificar la eliminación de la información.	OK
E-3.1	Verificar que la Aplicación sea compatible con el resto de las Aplicaciones del Ambiente de Pruebas.	OK
E-3.2	Determinar y validar aspectos técnicos (Tablas, Bases de Datos, perfiles , etc)	OK
E-3.3	Verificar correcto acceso a sub módulos o Programas	OK
E-4.1	Verificar el correcto formato de los informes generados	OK
E-4.2	Verificar que se esté siguiendo una numeración secuencial lógica de acuerdo a un orden establecido.	OK
E-4.3	Verificar que se realice el proceso de emisión del informe	OK
E-5.1	En listados revisar márgenes, verificando que el ancho del listado no sobrepase la hoja.	NA
E-5.2	Verificar que el formato de la información que aparezca en las impresiones sea comprensible.	NA
E-6.1	Verificar comportamiento al reanudar la aplicación desde segundo plano	OK
E-6.2	Verificar que el sistema maneje adecuadamente la pérdida temporal de conexión	OK
E-6.3	Verificar compatibilidad con impresión de tickets de soporte físico	NA

13.2 Resultados por componente

Tabla 24. Resultado global por componente

Componente	Casos aplicados	Resultado global	Observación QA
auth-service	8	8/8 OK	Componente validado sin error bloqueante.
profile-service	3	3/3 OK	Componente validado sin error bloqueante.
jobs-service	5	5/5 OK	Componente validado sin error bloqueante.
matching-service	2	2/2 OK	Componente validado sin error bloqueante.
frontend web	2	2/2 OK	Componente validado sin error bloqueante.
Redis / frontend	2	2/2 OK	Componente validado sin error bloqueante.
frontend	7	7/7 OK	Componente validado sin error bloqueante.
android-app	35	34/35 OK	Requiere revisión de casos no marcados.
backend	8	8/8 OK	Componente validado sin error bloqueante.

Los componentes principales fueron validados de forma separada y, posteriormente, dentro del flujo integrado del producto. Esta forma de trabajo permite detectar errores por módulo y disminuir el riesgo de que una falla aislada afecte la presentación final.

13.3 Evidencias visuales de testing

Las siguientes evidencias corresponden a capturas utilizadas durante el proceso de testing. Se incluyen como respaldo visual de ejecución de pruebas, revisión de servicios, validación de código, consola y comportamiento de componentes.

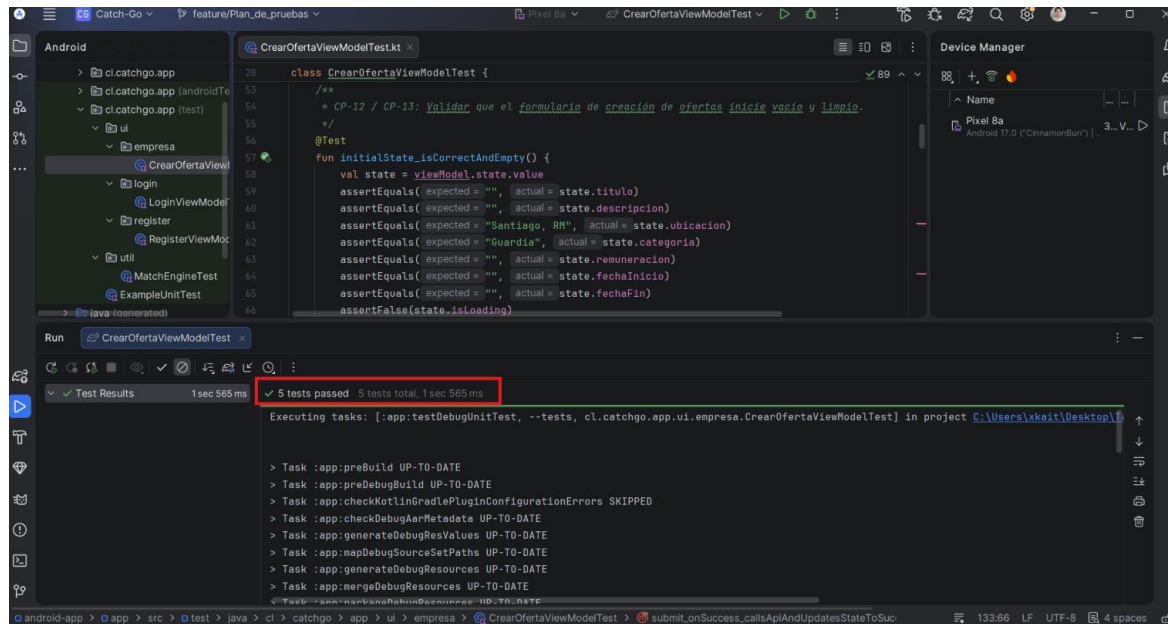


Figura 2. Evidencia de testing N° 1 del proyecto Catch & Go.

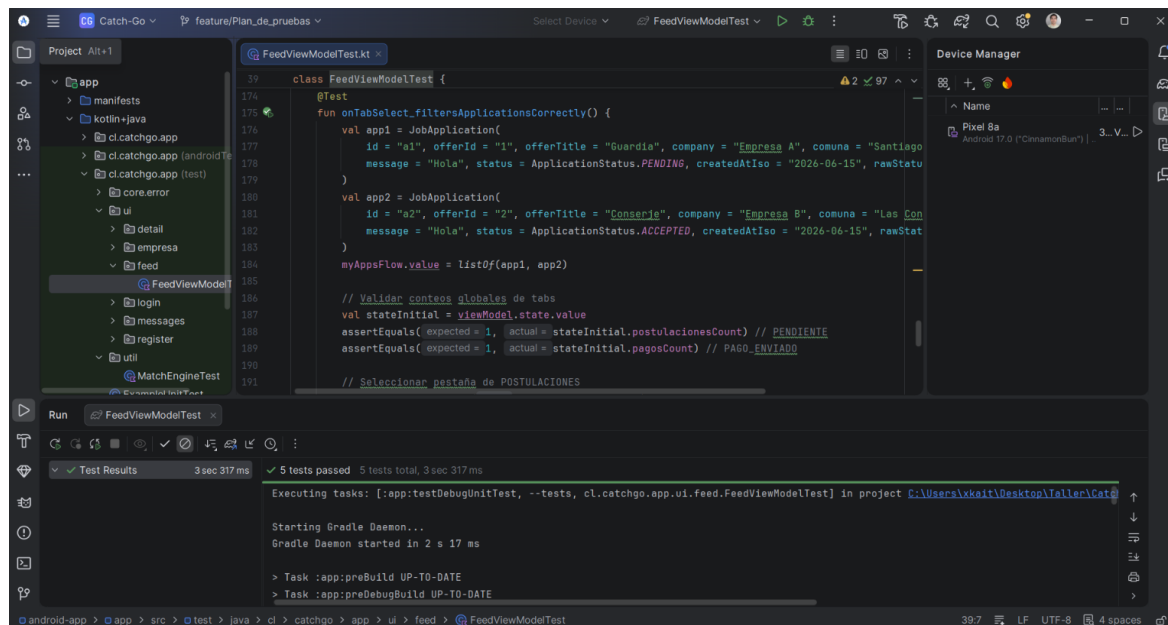


Figura 3. Evidencia de testing N° 2 del proyecto Catch & Go.

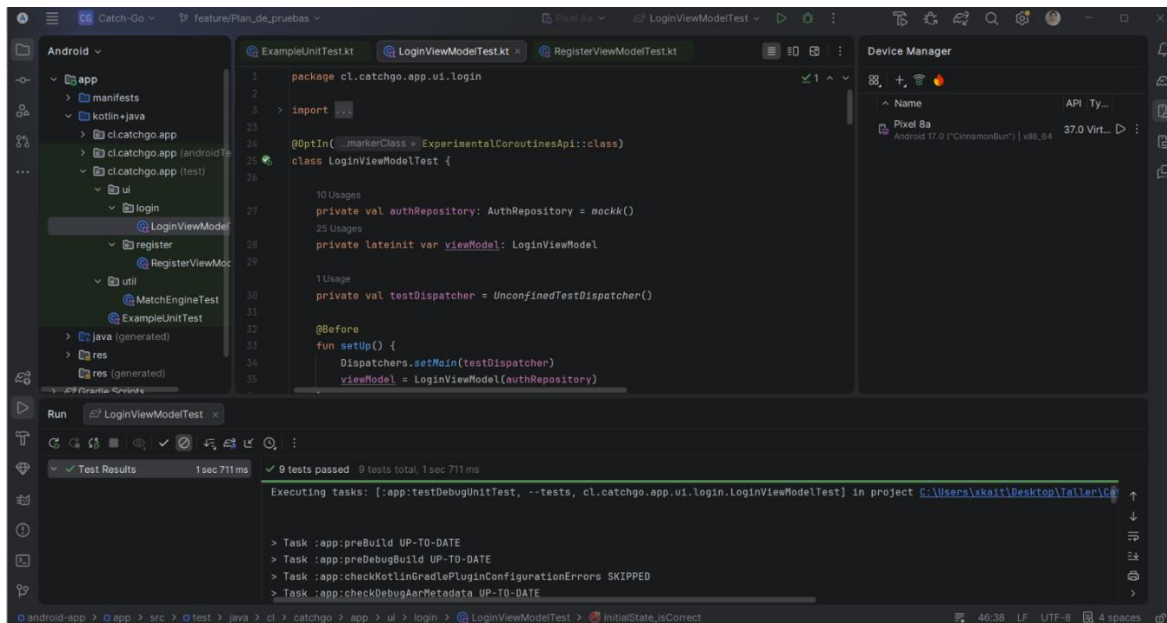


Figura 4. Evidencia de testing N° 3 del proyecto Catch & Go.

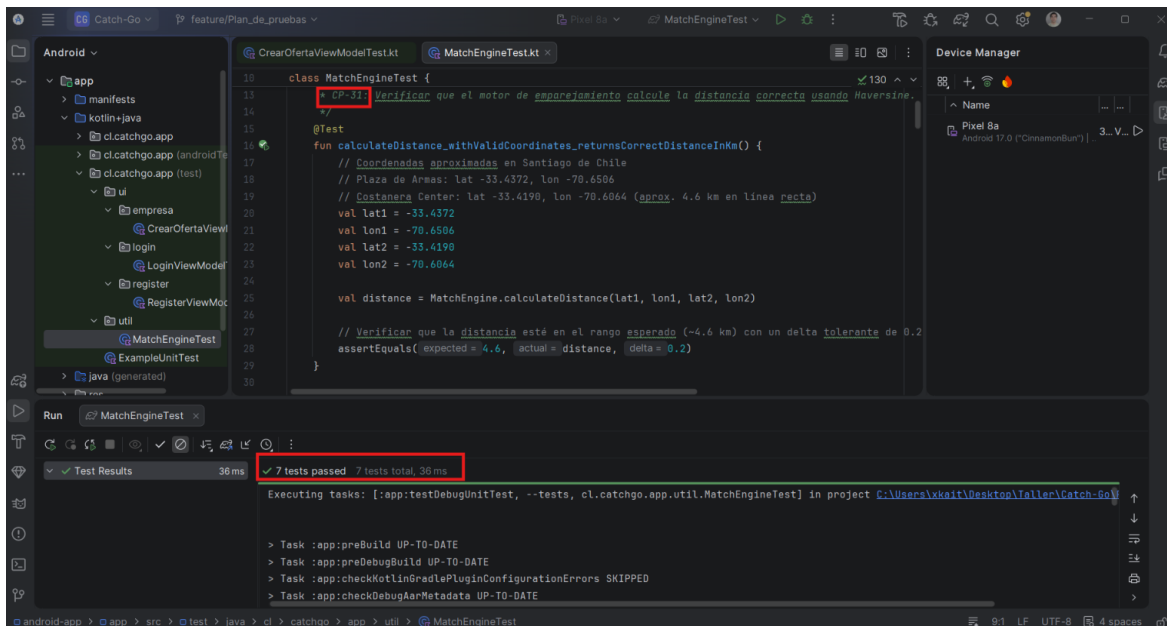


Figura 5. Evidencia de testing N° 4 del proyecto Catch & Go.

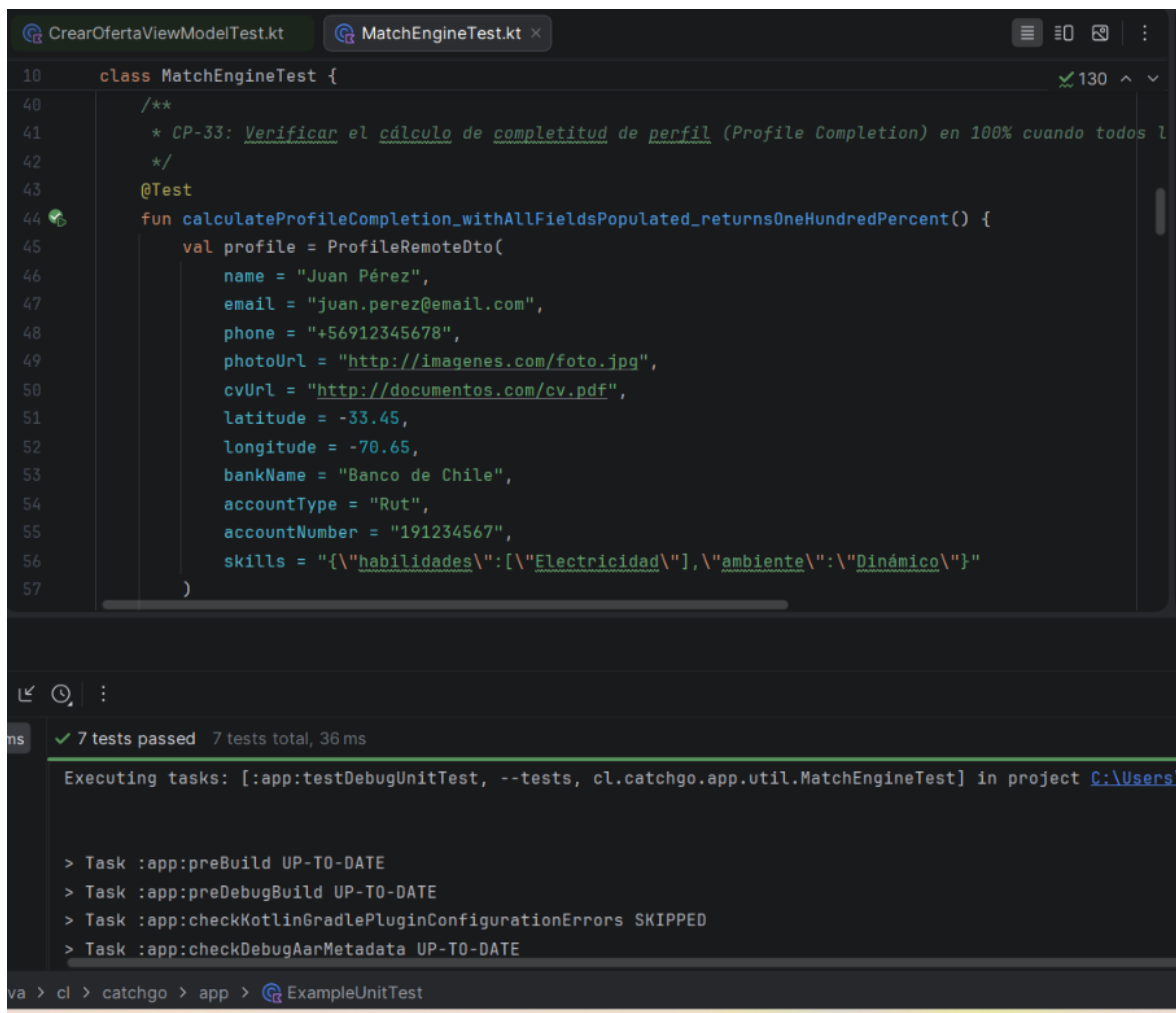


Figura 6. Evidencia de testing N° 5 del proyecto Catch & Go.

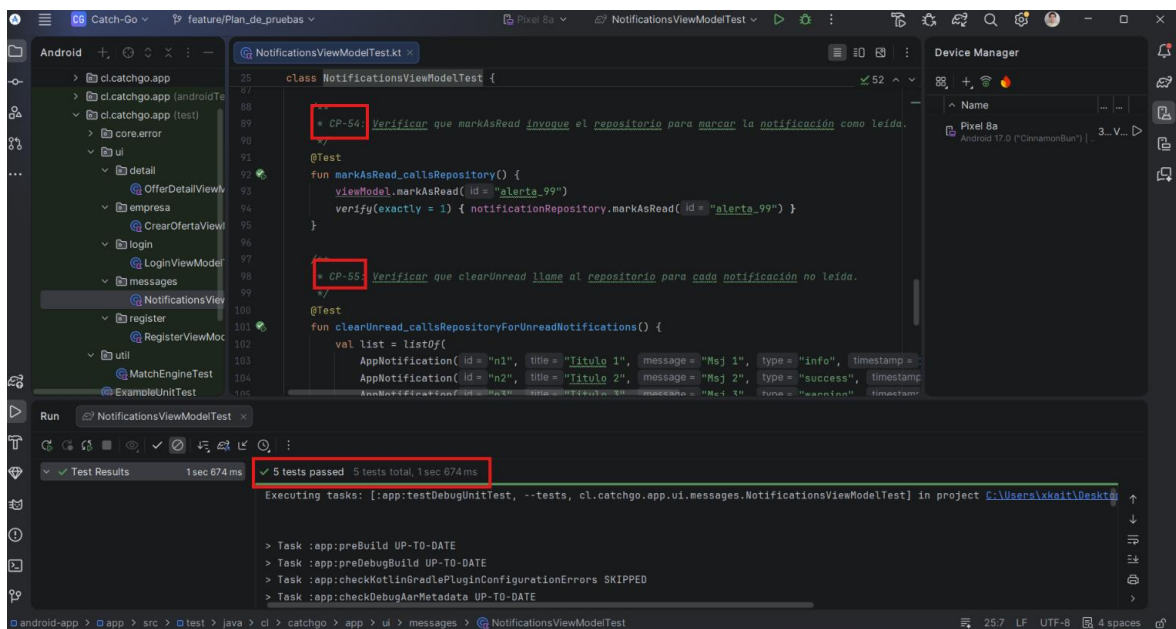


Figura 7. Evidencia de testing N° 6 del proyecto Catch & Go.

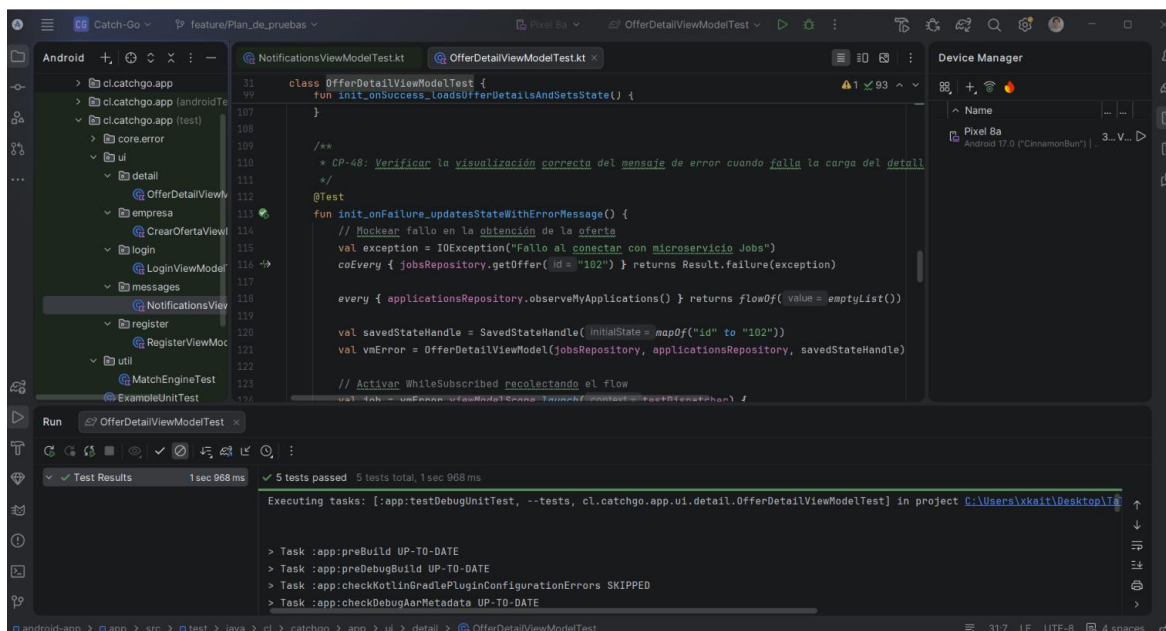


Figura 8. Evidencia de testing N° 7 del proyecto Catch & Go.

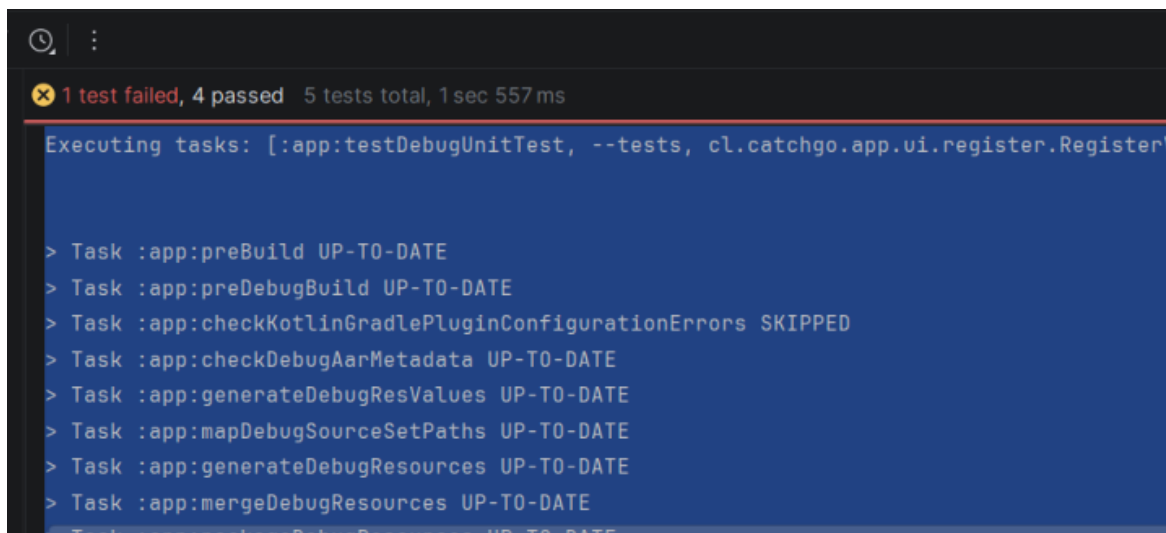


Figura 9. Evidencia de testing N° 8 del proyecto Catch & Go.

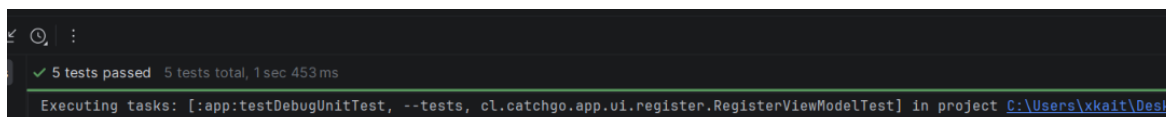


Figura 10. Evidencia de testing N° 9 del proyecto Catch & Go.

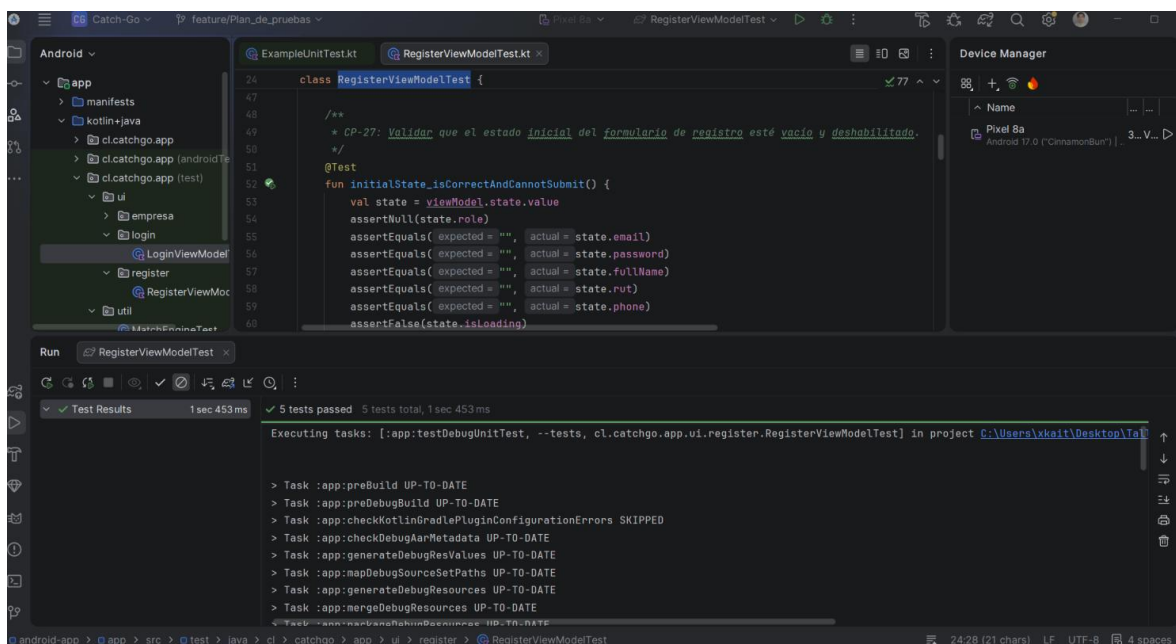


Figura 11. Evidencia de testing N° 10 del proyecto Catch & Go.

14. Mejoras realizadas según resultados de pruebas

Las mejoras aplicadas surgen directamente de los resultados de las pruebas. El objetivo fue corregir comportamientos que podían afectar la calidad del producto, reforzar validaciones, mejorar la experiencia del usuario y disminuir riesgos de seguridad.

Tabla 25. Hallazgos, mejoras aplicadas y fundamento técnico

Hallazgo / riesgo detectado	Caso relacionado	Mejora aplicada	Estándar asociado	Estado	Fundamento técnico
Possible registro con campos incompletos o correo inválido.	CP-03, CP-04, CP-63	Se reforzaron validaciones de campos obligatorios, formato de correo y coincidencia de contraseñas.	Compleitud / Corrección	Aplicada	Evita datos inconsistentes y reduce errores de soporte en el alta de usuarios.
Riesgo de acceso indebido con credenciales incorrectas.	CP-07	Se validó bloqueo de login y mensaje controlado de credenciales incorrectas.	Seguridad	Aplicada	Impide ingreso no autorizado y mantiene respuesta clara para el usuario.
Persistencia de sesión dependiente de almacenamiento local.	CP-08, CP-64	Se validó almacenamiento de token y redirección correcta según rol.	Usabilidad / Seguridad	Aplicada	Mejora continuidad de uso sin perder control de sesión.
Token JWT susceptible a expiración o alteración.	CP-65 a CP-68	Se agregaron pruebas de generación, claims, firma alterada y expiración.	Seguridad	Aplicada	Asegura que solo tokens válidos sean aceptados por el backend.
Tarifa negativa o inválida en publicación de oferta.	CP-13	Se reforzó validación para bloquear tarifas menores o iguales a cero.	Corrección / Pertinencia	Aplicada	Evita ofertas con condiciones económicas inválidas.
Errores de red o servicio podrían confundir al usuario.	CP-22	Se validó mensaje amigable ante desconexión o falla temporal.	Usabilidad	Aplicada	Reduce frustración y guía al usuario a reintentar.
Pago manual sin comprobante o con parámetros faltantes.	CP-25, CP-69, CP-71	Se validó deshabilitar confirmación sin archivo y controlar campos obligatorios en backend.	Seguridad / Compleitud	Aplicada	Disminuye riesgo de pagos incompletos o solicitudes inválidas.
Dashboard podría mostrar opciones incorrectas según rol.	CP-19, CP-20, CP-23	Se validaron vistas diferenciadas y limpieza de sesión al cerrar.	Seguridad / Pertinencia	Aplicada	Evita exposición de funciones que no corresponden al rol.

Las mejoras más relevantes se relacionan con validaciones de formularios, control de credenciales, uso de token, mensajes de error, persistencia de sesión, comunicación con servicios y consistencia visual entre pantallas. Esto fortalece el producto y demuestra que el testing se utilizó como una herramienta real de mejora, no solo como un trámite documental.

15. Cumplimiento de estándares de calidad, ética y seguridad

Tabla 26. Cumplimiento de estándares de calidad, ética y seguridad

Estándar	Cómo se aborda en el proyecto	Evidencia / casos asociados	Resultado
Usabilidad	Mensajes claros, botones con función reconocible, navegación con retorno y dashboard por rol.	Pruebas estándar E-1.1 a E-1.3, CP-19 a CP-24.	Cumplido
Seguridad	Control de credenciales, JWT, expiración, firma alterada y no exposición de datos reales.	CP-05 a CP-08 y CP-65 a CP-68.	Cumplido
Complejidad	Cobertura de registro, login, perfil, ofertas, postulaciones, matching, pagos y Android.	72 casos de prueba documentados.	Cumplido
Corrección	Validación de entradas, salidas esperadas y respuestas HTTP correctas.	CP-03, CP-04, CP-13, CP-69 a CP-72.	Cumplido
Pertinencia	Pruebas enfocadas en la problemática real de trabajos rápidos y contratación temporal.	Flujos trabajador/empresa y matching.	Cumplido
Ética y privacidad	Uso de datos ficticios, sin credenciales productivas ni información personal real.	Base de datos de prueba y dataset ficticio.	Cumplido

Desde el punto de vista ético, se trabajó con datos ficticios para pruebas, evitando utilizar información personal o comercial real. Además, se mantiene la separación entre roles, se controlan credenciales y se documentan evidencias sin exponer datos sensibles. Desde la perspectiva de calidad, el proyecto aplica trazabilidad requisito-caso, control de errores, evidencia de ejecución y mejoras justificadas técnicamente.

16. Riesgos, restricciones y condiciones para el éxito

Tabla 27. Riesgos, impactos y tratamiento recomendado

Riesgo / supuesto	Impacto	Tratamiento recomendado
El equipo es de solo tres integrantes con dedicación parcial.	Sobrecarga y retrasos.	Congelar alcance el 19 de mayo; priorizar auth, perfiles, ofertas, matching y demo.
Ramas con funcionalidades no consolidadas.	La demo puede no reflejar todo el trabajo real.	Actualizar develop y qa; hacer PR por módulo desde feature/init-frontend.
Declarar Machine Learning sin modelo entrenado.	Observación docente por sobrealcance.	Presentar matching por scoring y roadmap de ML posterior.
Cloud free con límites.	Caídas o suspensión por uso/idle.	Mantener demo local con Docker como respaldo y controlar variables de entorno.
Datos personales y normativa laboral.	Riesgo legal si se escala sin control.	Términos, privacidad, consentimiento y asesoría antes de producción real.
Marketplace sin masa crítica.	Poca adopción inicial.	Piloto por comuna y rubro; validar con empresas reales.

El principal riesgo del proyecto es el alcance, ya que el equipo debe equilibrar desarrollo, documentación y pruebas en un período académico acotado. Por esto, se prioriza un MVP demostrable, con funcionalidades clave funcionando y respaldadas por evidencias de testing.

17. Acta de aceptación y cierre

Tabla 28. Acta de aceptación de pruebas

Rol	Nombre	Conformidad	Firma	Fecha
Jefe de proyecto	Nicolás Iturrieta Villalobos	Acepta ejecución y resultados del ciclo QA		18/06/2026
Analista testing	Marco Tassara Carrasco	Acepta casos y evidencias documentadas		18/06/2026
Analista testing	Miguel Arredondo Vargas	Acepta casos y evidencias documentadas		18/06/2026

17.1 Conclusión final

El Informe 3 de Catch & Go consolida la evolución del proyecto desde su definición inicial hasta la validación del producto mediante pruebas. La integración del Informe 2 con el informe de testing permite presentar una visión completa: problema, solución, requerimientos, diseño, arquitectura, planificación, valorización, pruebas, evidencias, mejoras y cierre.

Como resultado, se demuestra que Catch & Go es una solución coherente con la problemática identificada, ya que propone una forma más ordenada, trazable y segura de conectar empresas con trabajadores para trabajos temporales. Además, el ciclo QA evidencia que el equipo no solo construyó funcionalidades, sino que también validó su comportamiento y aplicó mejoras para aumentar la calidad del producto.

Para una siguiente etapa, se recomienda profundizar pruebas automatizadas, aumentar cobertura de seguridad, consolidar ambientes cloud, ampliar pruebas con usuarios reales controlados y mejorar los indicadores del matching. Con ello, el producto podría avanzar desde un MVP académico hacia una solución con mayor proyección técnica y comercial.

17.2 Lecciones aprendidas

- El testing debe planificarse desde los requerimientos, no al final del desarrollo.
- La trazabilidad entre requisito, caso de prueba y evidencia facilita defender técnicamente el proyecto.
- Los datos ficticios son necesarios para cumplir criterios éticos y proteger información sensible.
- La arquitectura modular facilita probar componentes por separado, pero exige controlar bien las integraciones.
- Un MVP debe priorizar estabilidad y claridad de flujo antes que agregar demasiadas funcionalidades.

18. Anexos

18.1 Fuentes y referencias internas utilizadas

Tabla 29. Fuentes y anexos del proyecto

Fuente	Dato utilizado	URL
Repositorio Catch & Go	Estructura, stack, ramas y alcance técnico	https://github.com/ProgramacionDuoc26/Catch-Go.git
Railway	Free trial, Hobby USD 5 mínimo de uso y Pro USD 20 mínimo de uso	https://railway.com/pricing
Vercel	Hobby gratis y Pro USD 20/mes + uso adicional	https://vercel.com/pricing
Supabase	Free tier y Pro referencial desde USD 25/mes + uso	https://supabase.com/pricing
Google Maps Platform	Geocoding/Geolocation con free usage cap de 10.000 y tarifa por 1.000 usos	https://developers.google.com/maps/billing-and-pricing/pricing
Cinco Días / El País	Job&Talent: ronda de EUR 92 millones y valoración EUR 1.300 millones	https://cincodias.elpais.com/companias/2025-04-10/jobtalent-cierra-una-ronda-de-92-millones-con-una-valoracion-de-1300-millones-y-da-entrada-a-blackrock.html
Excel entregado	Gantt, valorización, modelo de negocio, evaluación y referentes	Gantt Catch_and_go.xlsx

18.2 Pauta sugerida para defensa oral

Bloque	Qué explicar	Tiempo sugerido
Problema y solución	Explicar por qué Catch & Go responde a una necesidad real de trabajos rápidos y reclutamiento trazable.	2 minutos
Arquitectura y módulos	Mostrar backend, frontend, Android, base de datos y separación por servicios.	3 minutos
Plan de pruebas	Explicar estrategia, criterios de entrada/salida, datos de prueba y trazabilidad.	3 minutos

Resultados y evidencias	Mostrar resumen 72 casos, evidencias visuales y resultados por componente.	3 minutos
Mejoras y cierre	Explicar mejoras aplicadas, calidad, ética, seguridad y conclusión final.	2 minutos